

The Lasso Distribution: Properties, Sampling Methods, and Applications in Bayesian Lasso Regression

by Mohammad Javad Davoudabadi, Jonathon Tidswell, Samuel Muller, Garth Tarr and John T. Ormerod

Abstract In this paper, we introduce a new probability distribution, the Lasso distribution. We derive several fundamental properties of the distribution, including closed-form expressions for its moments and moment-generating function. Additionally, we present an efficient and numerically stable algorithm for generating random samples from the distribution, facilitating its use in both theoretical and applied settings. We also establish that the Lasso distribution belongs to the exponential family. A direct application of the Lasso distribution arises in the context of an existing Gibbs sampler, where the full conditional distribution of each regression coefficient follows this distribution. This leads to a more computationally efficient and theoretically grounded sampling scheme. To facilitate the adoption of our methodology, we provide an R package, *BayesianLasso*, available on CRAN, implementing the proposed methods. Our findings offer new insights into the probabilistic structure underlying the Lasso penalty and provide practical improvements in Bayesian inference for high-dimensional regression problems.

1 Introduction

The Lasso (Least Absolute Shrinkage and Selection Operator) regression method, introduced by Tibshirani (1996), has become a cornerstone in high-dimensional statistical modeling due to its ability to perform variable selection and regularization simultaneously. The Bayesian formulation of the Lasso has been extensively studied, often relying on a Laplace prior for the regression coefficients, which can be expressed as a scale mixture of a Gaussian distribution (Park and Casella, 2008). However, despite its widespread use, existing formulations often face computational challenges, particularly in efficiently sampling from full conditional distributions in a Gibbs sampling framework (Hans, 2009).

In this paper, we propose a new probability distribution, referred to as the Lasso distribution, which arises naturally in the Bayesian formulation of Lasso regression. We fully develop this distribution and derive several of its key properties, including moments, a moment-generating function, and an efficient, numerically stable method for sampling from it. Key to these derivations is the accurate and numerically stable evaluation of the Mills ratio that arises in several of these functions (Mills, 1926). Further, we establish that the Lasso distribution belongs to the class of exponential family distributions, making it a theoretically attractive choice for modeling and inference. It is important to note that our formulation of the Lasso distribution differs from the one implemented in the *LaplacesDemon* package (Statisticat and LLC., 2021), which instead corresponds to the scale mixture of normals prior used in Park and Casella (2008).

As an application of the Lasso distribution, we note that it arises naturally as part of a Gibbs sampling scheme, where each coefficient is sampled individually (see Hans, 2009). By leveraging the Lasso distribution as the full conditional distribution for the regression coefficients, we achieve a computationally efficient sampling scheme. To facilitate reproducibility and practical implementation, we provide an R package that implements our proposed methods.

The remainder of the paper is structured as follows. Section 2 outlines our Bayesian hierarchical model, which motivates the Lasso distribution. In Section 3, we introduce the Lasso distribution. In Section 3.1, we describe how to sample efficiently in a numerically stable manner from the Lasso distribution. Section 4 describes how to use the package *BayesianLasso*. An application of the Lasso distribution via Gibbs sampling is given in Section 5. We present a performance comparison on benchmark datasets in Section 6 and conclude with a brief discussion in Section 7.

2 The Bayesian Lasso

We consider the standard linear regression model $\mathbf{y}|\boldsymbol{\beta}, \sigma^2 \sim \mathcal{N}(\mathbf{X}\boldsymbol{\beta}, \sigma^2 \mathbf{I}_n)$ for the observed dataset $\mathcal{D} = \{\mathbf{y}, \mathbf{X}\}$, where $\mathbf{y}$ is an n -dimensional vector of centered responses, $\mathbf{X}$ is an $n \times p$ matrix of standardized predictors, $\boldsymbol{\beta}$ is a p -dimensional vector of regression coefficients, and σ^2 denotes the residual variance parameter. In practice, it is often desirable to perform variable selection alongside

parameter estimation, particularly when many covariates may be irrelevant. To this end, [Tibshirani \(1996\)](#) proposes Lasso penalized regression, which introduces an ℓ_1 penalty to encourage sparsity in the estimated coefficients. The regularization strength is governed by a non-negative tuning parameter λ . The prior structure

$$\beta_j | \sigma^2, \tau_j \sim N(0, \sigma^2 \tau_j), \quad \text{and} \quad \tau_j \stackrel{\text{iid}}{\sim} \text{Gamma}(1, \lambda^2/2), \quad 1 \leq j \leq p. \quad (1)$$

mimics this penalty when the auxiliary parameter τ_j is marginalized out ([Park and Casella, 2008](#)). This hierarchical representation allows for full posterior inference while promoting sparsity through the prior structure. Here, we adopt independent conjugate priors for σ^2 and λ^2 : $\sigma^2 \sim \text{IG}(\tilde{a}, \tilde{b})$ and $\lambda^2 \sim \text{Gamma}(\tilde{u}, \tilde{v})$ where $\tilde{a} > 0$, $\tilde{b} > 0$, $\tilde{u} > 0$, and $\tilde{v} > 0$ are fixed prior hyperparameters.

We propose a modification to the hierarchical prior model in (1) that introduces a different auxiliary-variable representation. Specifically, instead of the local scale parameter τ_j used in (1), we introduce the variable η_j and express the prior for β_j through the alternative hierarchical formulation

$$\beta_j | \sigma^2, \eta_j \sim N\left(0, \frac{\sigma^2}{\eta_j \lambda^2}\right), \quad \text{and} \quad \eta_j \stackrel{\text{iid}}{\sim} \text{IG}(1, 1/2), \quad 1 \leq j \leq p. \quad (2)$$

This representation is not identical to (1) but is constructed so that the resulting full conditional distribution of β_j admits the kernel form given in (3), which facilitates the sampling strategy used in our Gibbs sampler.

[Hans \(2009\)](#) takes a different approach. Instead of using the auxiliary variable representations in (1) and (2), the standard Gibbs sampler of [Hans \(2009\)](#) samples from the full conditional distributions of the parameters individually, employing a weighted combination of two truncated normal distributions. It can be shown that the full conditional distribution for each β_j is proportional to

$$p(\beta_j | \mathcal{D}, \beta_{-j}, \sigma^2, \lambda^2) \propto \exp\left(-\frac{1}{2}a\beta_j^2 + b\beta_j - c|\beta_j|\right) \quad (3)$$

where a , b , and c are constants that depend on $\mathcal{D}$, β_{-j} , σ^2 , and λ^2 . This kernel corresponds to the weighted combination of two truncated normal distributions utilized by [Hans \(2009\)](#). Further details can be found in the Supplementary Material.

Although this conditional kernel can be expressed as a weighted mixture of truncated normal distributions and sampled within a Gibbs sampler ([Hans, 2009](#)), it has not previously been formalized as a standalone distribution with explicitly derived analytical properties. In particular, [Hans \(2009\)](#) utilized the truncated normal mixture representation for computational purposes but did not formally study the kernel as a distinct probability distribution, derive its properties in closed form, or develop numerically stable normalization formulas. To address this challenge, we develop a new distribution, which we term the *Lasso distribution*, arising naturally in the context of the Lasso regression model. We establish its analytical properties together with stable computational methods and an accompanying open-source R package implementation. In the next section, we formally define the Lasso distribution and derive several of its key properties, including its probability density function (PDF), cumulative distribution function (CDF), moment generating function (MGF), and moments.

3 The Lasso distribution

We derive several properties of the Lasso distribution, including the probability density function (PDF), cumulative density function (CDF), moment generating function (MGF), and moments. For detailed derivations, please refer to Section B of the Supplementary Materials.

A random variable X that maps from a probability space to the real line $\mathbb{R}$ has a Lasso distribution with parameters a , b , and c , denoted $X \sim \text{Lasso}(a, b, c)$, if its PDF is

$$p(X, a, b, c) = Z^{-1} \exp\left(-\frac{1}{2}aX^2 + bX - c|X|\right),$$

where $a \geq 0$, $b \in \mathbb{R}$, $c \geq 0$ (with a and c not both 0), and normalizing constant Z given by

$$Z(a, b, c) = \int_{-\infty}^{\infty} \exp\left(-\frac{1}{2}aX^2 + bX - c|X|\right) dx = \sigma \left[\frac{\Phi(\mu_1/\sigma)}{\phi(\mu_1/\sigma)} + \frac{\Phi(-\mu_2/\sigma)}{\phi(-\mu_2/\sigma)} \right],$$

with $\mu_1 = (b - c)/a$, $\mu_2 = (b + c)/a$, and $\sigma^2 = 1/a$. Here, $\Phi(\cdot)$ and $\phi(\cdot)$ denote the CDF and PDF of the standard normal distribution, respectively. The parameter a controls the quadratic curvature of the density and governs its overall concentration. The parameter b introduces a linear tilt, inducing

asymmetry and shifting mass toward positive or negative values. The parameter c controls the strength of the absolute value term, increasing concentration near zero and producing the characteristic cusp of the Lasso distribution. Evaluating Z can be numerically challenging due to potential overflow, underflow, or divide-by-zero problems. To address this, our R package **BayesianLasso** implements a numerically stable and efficient method for computing Z , automatically managing these numerical difficulties (Ormerod et al., 2025).

The Lasso distribution can be written as a mixture of two truncated normal distributions. As we shall see, for different extreme values of a , b and c , the Lasso distribution can behave like a normal distribution, a Laplace distribution, an asymmetric Laplace distribution, a positively truncated normal distribution, or a negatively truncated normal distribution. By characterizing the limiting behavior under extreme parameter values, the R package **BayesianLasso** leverages appropriate approximations to known distributions ensuring robust and efficient computation even in numerically challenging settings. Figure 1 shows the density of the univariate Lasso distribution for different parameter values.

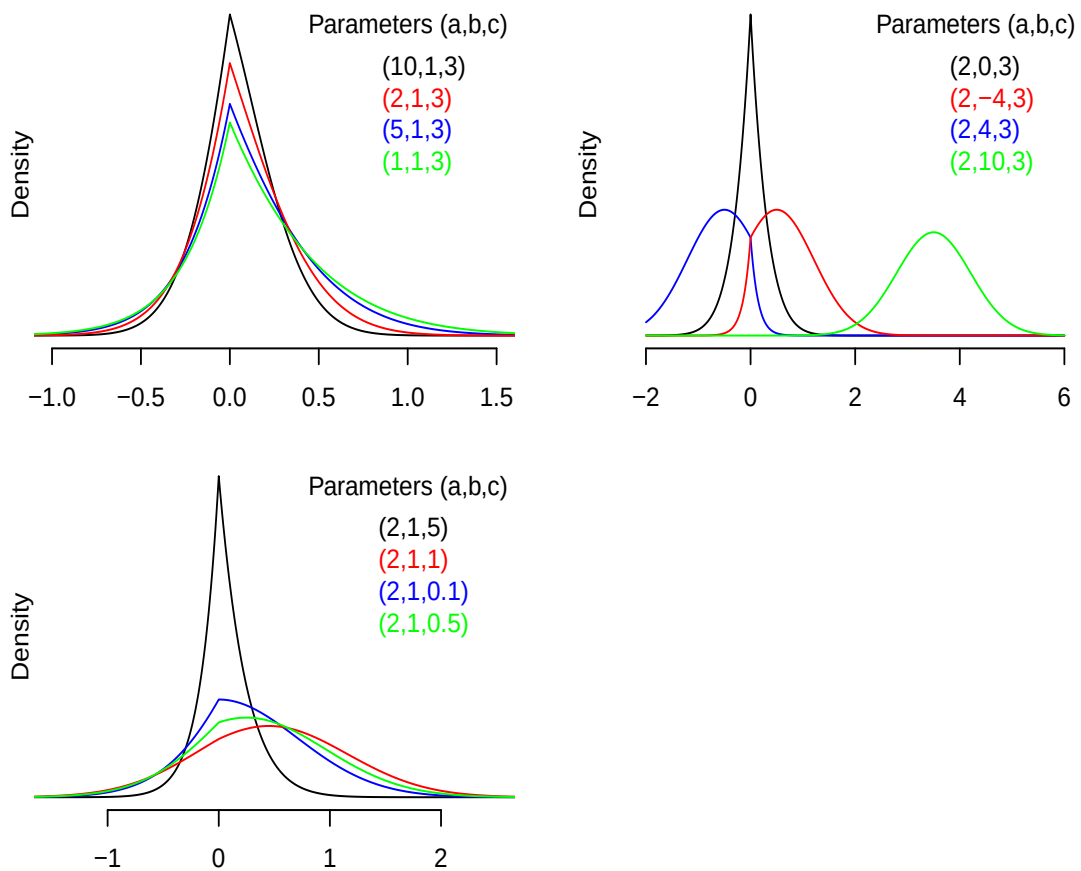


Figure 1: The Lasso density function is depicted for different parameter values. In the top left panel, parameter a is varied while the other parameters are fixed. In the top right and bottom left panels, parameters b and c are varied, respectively, while the other parameters are fixed.

The MGF of the Lasso distribution is $M(t) = Z(a, b + t, c) / Z(a, b, c)$ and the calculation of the moments of the Lasso distribution via the MGF is algebraically cumbersome. See Section B of the Supplementary Materials for the derivation of the MGF. However, by expressing the Lasso distribution as a mixture of two truncated normal distributions, the moments can be calculated as $\mathbb{E}(X^r) = (1 - w) \cdot \mathbb{E}(A^r) + w \cdot \mathbb{E}(B^r)$, where the mixing proportion is given by

$$w = \left[1 + \frac{\Phi(\mu_1/\sigma)}{\phi(\mu_1/\sigma)} \frac{\phi(-\mu_2/\sigma)}{\Phi(-\mu_2/\sigma)} \right]^{-1},$$

where $A \sim TN_+(\mu_1, \sigma^2)$ and $B \sim TN_-(\mu_2, \sigma^2)$ are the positively truncated and negatively truncated normal distributions, respectively.

The CDF of the Lasso distribution is given by

$$P(x) = \mathcal{P}(X < x) = \begin{cases} w \frac{\Phi\left(\frac{x-\mu_2}{\sigma}\right)}{\Phi(-\mu_2/\sigma)} & \text{if } x \leq 0 \text{ and} \\ 1 - (1-w) \frac{\Phi\left(\frac{\mu_1-x}{\sigma}\right)}{\Phi(\mu_1/\sigma)} & \text{if } x > 0. \end{cases}$$

The inverse CDF of the Lasso distribution is given by

$$P^{-1}(u) = \begin{cases} \mu_2 + \sigma \Phi^{-1}[\Phi(-\mu_2/\sigma)u/w] & \text{if } u \leq w \text{ and} \\ \mu_1 - \sigma \Phi^{-1}[\Phi(\mu_1/\sigma)(1-u)/(1-w)] & \text{if } u > w. \end{cases} \quad (4)$$

Finally, the mode of the Lasso distribution is equal to $\max(|b| - c, 0) \cdot \text{sign}(b)/a$, where $\text{sign}(\cdot)$ is the sign function.

The R package **BayesianLasso** enables efficient and numerically stable sampling from the Lasso distribution by evaluating the inverse CDF using uniform random draws. To ensure robustness, particularly in the tails, the implementation incorporates safeguards against numerical issues such as overflow and underflow. A key component of this approach is the numerically stable evaluation of the Mills ratio, defined as $m(x) = \Phi(-x)/\phi(-x)$, which is used in computing the normalizing constant. Specifically, the package computes

$$v_1 = \frac{c - |b|}{\sqrt{a}} \quad \text{and} \quad v_2 = \frac{c + |b|}{\sqrt{a}},$$

and evaluates the normalizing constant as $Z = [m(v_1) + m(v_2)] / \sqrt{a}$. This combination of numerical stability and computational efficiency underpins the practical value of the package and is a key reason for its robustness in high-dimensional Bayesian inference.

3.1 Sampling from the Lasso distribution

Key quantities required for various expressions in the previous section include the computation of Z and w . Additionally, special care must be taken when calculating the inverse CDF to prevent both overflow and underflow. To address these concerns, we define $v_1 = (c - |b|) / \sqrt{a}$ and $v_2 = (c + |b|) / \sqrt{a}$. The normalizing constant Z can then be expressed as $Z = [m(v_1) + m(v_2)] / \sqrt{a}$. For negative inputs, i.e., $x < 0$, the Mills ratio is $m(x) = \phi(x)^{-1} - m(-x)$. Since $v_2 \geq 0$ and $v_1 \geq 0$ when $|b| \leq c$, we can partition the definition of Z into two cases

$$Z = \begin{cases} \frac{1}{\sqrt{a}} [m(v_1) + m(v_2)] & |b| \leq c \text{ and} \\ \frac{1}{\sqrt{a}} \left[\left(\phi(v_1)^{-1} - m(-v_1) \right) + m(v_2) \right] & |b| > c; \end{cases}$$

where both alternatives have the requirement for two Mills ratio calculations for positive parameters; this is a potential use case for a SIMD, which is short for single instruction/multiple data, vectorized implementation of the positive side of the Mills ratio.

The second component of Z (for $|b| > c$) has an obvious potential overflow when $|v_1|$ is large. In such cases, it may be necessary to transition to logarithmic space to represent $\phi(v_1)$ to avoid underflow/overflow. However, unnecessary logarithmic computations should be avoided, as they are computationally expensive and can reduce numerical precision when not required. Marsaglia (2004) notes that the precision of evaluating the standard normal density function $\phi(x)$ and its tail probability deteriorates as x increases, due to the limitations of computing $\exp(-x^2/2)$ in double-precision arithmetic. While the paper primarily examines values up to $x \approx 16$, it is also known that $\phi(x)$ effectively loses all precision for $x \geq 37$, where the exponential term underflows to zero and the reciprocal overflows. Now, consider the case where $v_1 < -37$, making overflow a concern. By definition, we have $37 < -v_1 < v_2$, which implies $m(37) > m(-v_1) > m(v_2) > 0$. Since $m(37) \approx 0.027$, it follows that $-0.027 < m(v_2) - m(-v_1) < 0$. Within the limits of double precision, this difference becomes negligible due to rounding errors for $v_1 \approx -9$, and certainly for $v_1 < -37$, where $\phi(-37) < 6 \times 10^{-300}$. Consequently, Z can be approximated as

$$Z \approx \frac{1}{\sqrt{a}} \left[\phi(v_1)^{-1} \right].$$

Thus, we can safely use the approximation $m(v_1) \approx \phi(v_1)^{-1}$ in such cases.

For a fast and accurate approximation of Mills ratio for positive arguments, we employ a degree (8,9) rational polynomial optimized using the Remez algorithm (Reemtsen, 1990) over the interval [0,600], achieving an accuracy of up to 12 significant figures. The leading coefficients of both the numerator and denominator are positive, ensuring that the approximation asymptotically converges to a small fraction. This stability allows the relative error to remain consistent, preserving 11 significant figures up to approximately $x \approx 2000$. Beyond this point, the precision gradually improves, as Mills ratio satisfies $m(x) \sim 1/x$ for large x .

Let

$$\begin{aligned} p_0 &= 46697.7602201933, & p_1 &= 69339.6909002865, & p_2 &= 50590.6980372328, \\ p_3 &= 23184.62760379742, & p_4 &= 7236.31450136984, & p_5 &= 1572.136841909630, \\ p_6 &= 232.9967987466022, & p_7 &= 21.74833514806325, & \text{and } p_8 &= 1.000000000000095. \end{aligned}$$

and let

$$\begin{aligned} q_0 &= 37259.42190376593, & q_1 &= 85053.78630172011, & q_2 &= 89598.92885811838, \\ q_3 &= 57370.93777717682, & q_4 &= 24713.27114352290, & q_5 &= 7467.311205544661, \\ q_6 &= 1593.885178714749, & q_7 &= 233.9967987305447, & q_8 &= 21.74833514813385, \end{aligned}$$

and $q_9 = 1$. Let

$$\tilde{m} = \frac{p_0 + x(p_1 + x(p_2 + x(p_3 + x(p_4 + x(p_5 + x(p_6 + x(p_7 + xp_8))))))}{q_0 + x(q_1 + x(q_2 + x(q_3 + x(q_4 + x(q_5 + x(q_6 + x(q_7 + x(q_8 + x)))))))).$$

Our approximation of $m(x)$ for $x > 0$ is then

$$m(x) = \begin{cases} \tilde{m} & x < 1.75 \times 10^{34} \\ 1/x & \text{otherwise.} \end{cases}$$

Next, we define

$$w_1 = \frac{1}{1 + m(v_1) m(v_2)^{-1}} \quad \text{and} \quad w_2 = \frac{1}{1 + m(v_1)^{-1} m(v_2)}.$$

As $v_1 \rightarrow -\infty$, it follows that $m(v_1) \rightarrow \infty$, leading to $w_1 \rightarrow 0$ and $w_2 \rightarrow 1$. However, due to the dynamic range limitations of floating-point representation, w_1 is representable, whereas w_2 rounds to 1 for $v_1 \lesssim -5.5$, resulting in a loss of precision. Since floating-point arithmetic offers much better precision near zero than near one, using w_2 when $v_1 \ll 0$ can introduce numerical errors. To mitigate this, we define

$$w = \begin{cases} w_1 & \text{if } b \leq 0 \text{ and} \\ w_2 & \text{if } b > 0, \end{cases} \quad \text{and} \quad 1 - w = \begin{cases} w_2 & \text{if } b \leq 0 \text{ and} \\ w_1 & \text{if } b > 0. \end{cases}$$

Thus, a simple yet numerically stable rule is to work with w when $b \leq 0$ and with $1 - w$ when $b > 0$.

The computation of $P^{-1}(u)$ is carried out in four distinct cases, determined by whether $u \leq w$ and whether $b > 0$. First, the arguments of $\Phi^{-1}(\cdot)$ are carefully computed to prevent overflow. In extreme cases, underflow may still occur. When underflow is detected, the arguments of $\Phi^{-1}(\cdot)$ are instead computed on the logarithmic scale, and the asymptotic tail formula of Mächler (2022) is used to evaluate $\Phi^{-1}(\cdot)$ accurately.

4 Using the BayesianLasso package

To illustrate the functionality of the **BayesianLasso** package, we provide examples of its key functions, including density evaluation, cumulative distribution, random sampling, and quantile calculations.

4.1 Probability density function (PDF)

The function `dlasso()` computes the probability density function of the Lasso distribution for given parameters. The following example plots the density of a Lasso(2,1,3) distribution, shown as a solid black line in Figure 2.

```
library(BayesianLasso)
```

```
# Define parameters
```

```

a <- 2
b <- 1
c <- 3
x <- seq(-3, 3, length = 1000)

# Plot the density of Lasso(2,1,3)
plot(x, dlasso(x, a, b, c, logarithm = FALSE), type = 'l',
      xlab = "x", ylab = "Density",
      ylim = c(0, 2.25), yaxt = "n", bty = "n")

```

4.2 Cumulative distribution function (CDF)

The function `plasso()` calculates the cumulative distribution function of the Lasso distribution. Below, we compute the CDF at $x = -1$ for a $\text{Lasso}(2,1,3)$ distribution:

```

CDF_value <- plasso(-1, a, b, c)
print(CDF_value)
[1] 0.00176594

```

4.3 Random sampling from the Lasso distribution

To generate random samples from the Lasso distribution, we use `rlasso()`. The following example generates 100,000 samples and compares the empirical density with the theoretical density, as shown in Figure 2.

```

# Generate random samples
samples <- rlasso(100000, a, b, c)

# Compare empirical and theoretical density
plot(x, dlasso(x, a, b, c, logarithm = FALSE), type = "l", lty = 1,
      xlab = "x", ylab = "Density",
      xlim = c(-2, 2), yaxt = "n", bty = "n")
lines(density(samples), col = "red", type = "l", lty = 2)

legend("topright", legend = c("Theoretical", "Empirical"), col = c("black", "red"),
      lty = c(1, 2), bty = "n")

```

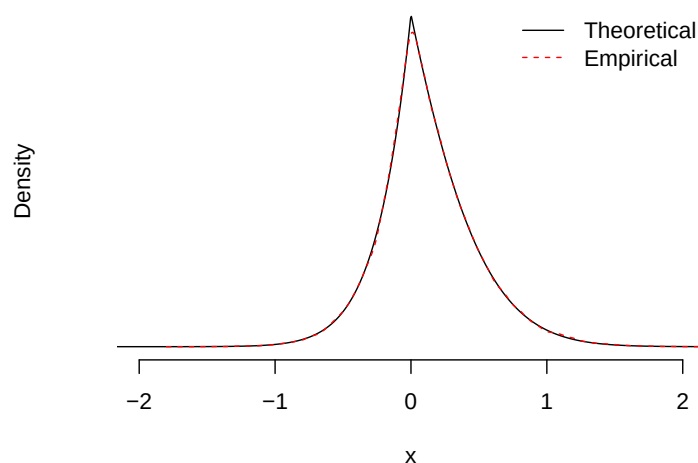


Figure 2: Theoretical and empirical densities for $\text{Lasso}(2,1,3)$. The theoretical density plot of $\text{Lasso}(2,1,3)$ using `dlasso(x, a, b, c, logarithm = FALSE)` is shown as a solid black line, and the empirical density is shown as a dashed line.

To assess implementation accuracy, we compare empirical moments obtained from Monte Carlo simulation with the theoretical mean, `elasso(a,b,c)`, and variance, `vlasso(a,b,c)`, derived in Sec-

tion 3. For multiple parameter configurations (a, b, c) chosen to represent different scales and degrees of concentration, 10^5 independent samples were generated using `rlasso()`. The empirical mean and variance were computed and compared with their analytical counterparts, and the absolute errors between theoretical and empirical moments are reported in Table 1.

(a, b, c)	Mean			Variance		
	Theoretical	Empirical	Error	Theoretical	Empirical	Error
(2, 1, 3)	0.121830	0.122919	0.001088	0.128773	0.129729	0.000955
(4, 4, 1)	0.773432	0.773007	0.000425	0.228095	0.226861	0.001233
(1, 1, 1)	0.503222	0.503450	0.000227	0.558956	0.559068	0.000111

Table 1: Comparison of theoretical and empirical moments for selected (a, b, c) parameter settings.

Table 1 demonstrates close agreement between the theoretical and empirical moments across all tested parameter settings. For 10^5 independent Monte Carlo draws, the empirical means and variances match their analytical counterparts to three or more decimal places, with absolute errors uniformly below 10^{-3} . The magnitude of these discrepancies is consistent with expected Monte Carlo error, indicating that the random number generator `rlasso()` and the analytical moment functions `elasso()` and `vlasso()` are correctly implemented and numerically stable.

4.4 Quantile function

The inverse-CDF sampler corresponding to Equation (4) is implemented in C++ which is located in `src/lasso_distribution.cpp`, and is exposed to users via the exported Rcpp interface `qlasso()`. The required evaluations of the normal cumulative distribution function and its inverse are performed using the R math library routines `R::pnorm5` and `R::qnorm5`. These routines compute the normal CDF and quantile using piecewise rational approximations with separate central and tail expansions, ensuring high numerical accuracy and stability even for extreme probability values.

Below, we compute the quantiles for probability values $p = \{0.1, 0.3, 0.6\}$:

```
p_values <- c(0.1, 0.3, 0.6)
quantiles <- qlasso(p_values, a, b, c)
print(quantiles)
[1] -0.28183916 -0.04935763 0.16137104
```

4.5 Numerical stability and boundary behaviour

Failure cases of naive implementations. A direct implementation of the Lasso distribution based on the density

$$f(x \mid a, b, c) \propto \exp\left(-\frac{1}{2}ax^2 + bx - c|x|\right)$$

can suffer from substantial numerical instability. In particular, naive evaluation of the normalizing constant may overflow when $|b|/c$ is large, due to the exponential terms arising from Gaussian tail probabilities. Similarly, direct subtraction of nearly equal floating-point quantities in cumulative distribution function (CDF) computations may lead to catastrophic cancellation in extreme tail regions.

In addition, straightforward inversion of the CDF for quantile evaluation can fail without careful bracketing, particularly in highly skewed parameter regimes. These issues are common in naive implementations that rely solely on closed-form expressions without attention to floating-point behaviour.

The implementation in `BayesianLasso` mitigates these problems by performing normalization and tail probability calculations on the log-scale, separating positive and negative domains explicitly, and using numerically stable bracketing procedures for quantile inversion. These design choices ensure stable behaviour across a wide range of parameter values.

Boundary and near-degenerate regimes. We further investigated behaviour in limiting parameter regimes:

- $c \rightarrow 0$. In this limit, the Lasso distribution converges to a Gaussian distribution with mean b/a and variance $1/a$. Numerical experiments demonstrate that the implementation transitions smoothly to this Gaussian case without discontinuity or instability.

- $a \rightarrow 0$. As a approaches zero, the quadratic term vanishes and the distribution approaches a Laplace-type density. Direct evaluation of normalization constants becomes ill-conditioned in this regime; however, the log-scale implementation maintains numerical stability and produces accurate density and distribution evaluations.
- **Large $|b|/c$.** When $|b|/c$ is large, the distribution becomes highly skewed and naive exponential evaluations may overflow. The current implementation remains stable under such extreme skewness due to its log-scale normalization and careful handling of Gaussian tail probabilities.

These investigations demonstrate that the proposed implementation is robust not only in regular parameter settings but also in boundary and numerically challenging regimes. Such stability is essential for reliable use of the Lasso distribution in Bayesian computation and simulation-based inference.

In the next section, we incorporate the Lasso distribution into a Gibbs sampling algorithm as the full conditional distribution for the coefficients of the Lasso regression model.

5 Application of the Lasso distribution in Gibbs sampling

We modify the Gibbs sampler of [Hans \(2009\)](#) (henceforth Hans) by using the Lasso distribution as the full conditional distribution of the regression coefficients. We also consider a slightly modified version of the Gibbs sampler of [Park and Casella \(2008\)](#) (henceforth PC), as the alternative Gibbs sampler, by changing the representation of the auxiliary variable from (1) to (2).

Furthermore, these Gibbs samplers are based on the prior $\lambda^2 \sim \text{Gamma}(\tilde{u}, \tilde{v})$ which differs from [Hans \(2009\)](#), where the prior is placed on λ (which is conjugate) rather than λ^2 (which is not conjugate in the modified Hans sampler). The primary motivation for this choice is to ensure consistency with the parameterization used in the PC Gibbs sampler, which is formulated in terms of λ^2 . This allows us to place both samplers under the same underlying model specification and enables a direct and fair comparison between their sampling strategies. In addition, modeling λ^2 aligns naturally with the formulation of the Lasso distribution introduced in this paper, where λ^2 enters directly into the quadratic form of the conditional density. This parameterization therefore provides a more coherent connection between the model and the proposed sampling approach. However, unlike the conjugate specification for λ in [Hans \(2009\)](#), placing the prior on λ^2 breaks conjugacy, necessitating the use of a slice sampler ([Neal, 2003](#)) for its full conditional distribution.

Theoretically, both specifications induce similar shrinkage behavior, but they differ in computational properties. The conjugate prior leads to simpler Gibbs updates, whereas the non-conjugate specification requires auxiliary sampling steps. In practice, this may affect mixing for λ^2 , leading to poorer mixing in some settings (e.g., the Crime dataset in Table 5), which we regard as a computational trade-off of the chosen parameterization. All methods are implemented in the same programming language and executed on the same computer, ensuring that the primary distinction lies in the choice of Gibbs samplers used to fit the model.

5.1 Hans Gibbs sampler

As stated earlier, the standard Gibbs sampler proposed by [Hans \(2009\)](#) relies on the fact that the full conditional distribution for each β_j is given by (3), and [Hans \(2009\)](#) notes that $p(\beta_j | \mathcal{D}, \beta_{-j}, \sigma^2, \lambda^2)$ can be represented as a mixture of two truncated normal distributions. This is not a well-known distribution to sample. However, we show care needs to be taken to avoid numerical problems using this representation.

To facilitate sampling from the kernel in (3), we utilize the Lasso distribution ($\text{Lasso}(a, b, c)$), which we introduced in Section 3. We also employ the efficient and numerically stable sampling method developed in Section 3.1 to draw samples from this distribution. Notably, while the Hans sampler is specifically designed for the Lasso distribution, it is difficult to extend to other response types or alternative penalty structures.

Algorithm 1 presents our modified version of Hans' Gibbs sampling algorithm, where we model $\lambda^2 \sim \text{Gamma}(\tilde{u}, \tilde{v})$ instead of $\lambda \sim \text{Gamma}(\tilde{u}, \tilde{v})$. Unlike the original method in [Hans \(2009\)](#), which employs conjugate sampling for λ and rejection sampling for σ^2 , our approach collapses the auxiliary variable η_j to improve mixing and utilizes slice sampling for both $\sigma^2 | \mathcal{D}, \beta, \lambda^2$ and $\lambda^2 | \mathcal{D}, \beta, \sigma^2$ ([Neal, 2003](#)). Notably, the same slice sampler is applied to both parameters, as their full conditional distributions are inverse transformations of each other.

In Algorithm 1, the quantity $\mathbf{r}_j$ is analogous to the partial residuals used in coordinate descent methods for optimizing the Lasso objective function (see, e.g., [Hastie et al., 2015](#), Section 5.4.2).

Algorithm 1 Modified Hans Gibbs sampler

```

1: Inputs:  $\mathbf{y}, \mathbf{X}, \tilde{a} > 0, \tilde{b} > 0, \tilde{u} > 0, \tilde{v} > 0, \boldsymbol{\beta}^{(0)} = \mathbf{0}_p, \sigma^{2(0)} > 0, \lambda^{(0)} > 0;$ 
2: for  $i = 1, \dots, N$  do
3:    $\boldsymbol{\beta}^{(i)} \leftarrow \boldsymbol{\beta}^{(i-1)};$ 
4:   if  $n > p$  then
5:      $\boldsymbol{\delta} \leftarrow (\mathbf{X}^T \mathbf{X}) \boldsymbol{\beta}^{(i)};$ 
6:     for  $j = 1, \dots, p$  do
7:        $\boldsymbol{\delta} \leftarrow \boldsymbol{\delta} - \mathbf{X}_j \boldsymbol{\beta}_j^{(i)};$ 
8:        $\boldsymbol{\beta}_j^{(i)} \sim \text{Lasso} \left( \frac{\|\mathbf{X}_j\|_2^2}{\sigma^{2(i-1)}}, \frac{(\mathbf{X}^T \mathbf{y})_j - \delta_j}{\sigma^{2(i-1)}}, \frac{\lambda^{(i-1)}}{\sigma^{(i-1)}} \right);$ 
9:        $\boldsymbol{\delta} \leftarrow \mathbf{r} + \mathbf{X}_j \boldsymbol{\beta}_j^{(i)};$ 
10:    end for
11:     $\text{RSS} \leftarrow \|\mathbf{y}\|_2^2 - 2\mathbf{y}^T \mathbf{X} \boldsymbol{\beta}_j^{(i)} + (\boldsymbol{\beta}_j^{(i)})^T \mathbf{X}^T \mathbf{X} \boldsymbol{\beta}_j^{(i)};$ 
12:  else
13:     $\mathbf{y} \leftarrow \mathbf{X} \boldsymbol{\beta}^{(i)};$ 
14:    for  $j = 1, \dots, p$  do
15:       $\mathbf{r}_j \leftarrow \mathbf{y} - (\hat{\mathbf{y}} - \mathbf{X}_j \boldsymbol{\beta}_j^{(i-1)});$ 
16:       $\boldsymbol{\beta}_j^{(i)} \sim \text{Lasso} \left( \frac{\|\mathbf{X}_j\|_2^2}{\sigma^{2(i-1)}}, \frac{\mathbf{X}_j^T \mathbf{r}_j}{\sigma^{2(i-1)}}, \frac{\lambda^{(i-1)}}{\sigma^{(i-1)}} \right);$ 
17:       $\hat{\mathbf{y}} \leftarrow \hat{\mathbf{y}} + \mathbf{X}_j (\boldsymbol{\beta}_j^{(i)} - \boldsymbol{\beta}_j^{(i-1)});$ 
18:    end for
19:     $\text{RSS} \leftarrow \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2;$ 
20:  end if
21:  Use slice sampling to sample  $\sigma^{2(i)}$  from the density
    
$$p(\sigma^2 | \mathcal{D}, \boldsymbol{\beta}^{(i)}, \lambda^{2(i-1)}) \propto \exp \left[ \left( \tilde{a} + \frac{n+p}{2} + 1 \right) \log(\sigma^2) - \left( \tilde{b} + \frac{1}{2} \text{RSS} \right) \sigma^{-2} - \frac{\lambda^{(i-1)}}{\sigma} \|\boldsymbol{\beta}^{(i)}\|_1 \right];$$

22:  Use slice sampling to sample  $\lambda^{2(i)}$  from the density
    
$$p(\lambda^2 | \mathcal{D}, \boldsymbol{\beta}^{(i)}, \sigma^{2(i)}) \propto \exp \left[ \left( \tilde{u} + \frac{p}{2} - 1 \right) \log(\lambda^2) - \tilde{v} \lambda^{2(i-1)} - \frac{\lambda}{\sigma^{(i)}} \|\boldsymbol{\beta}^{(i)}\|_1 \right].$$

23: end for

```

However, rather than computing the mode of $\beta_j | \mathcal{D}, \boldsymbol{\beta}_{-j}, \sigma^2$ as done in coordinate descent, we draw samples from its distribution.

Hans (2009) does not discuss the computational costs of the Gibbs samplers presented there. In contrast, our Algorithm 1 avoids matrix inversion, and as shown in Algorithm 1, sampling each variable requires $\mathcal{O}(\min(n, p))$ time, assuming the quantities $\mathbf{X}^T \mathbf{X}$ (for $n > p$), $\text{diag}(\mathbf{X}^T \mathbf{X})$ and $\mathbf{X}^T \mathbf{y}$ are computed outside the main loop. Moreover, Algorithm 1 accommodates both the $n > p$ and $p > n$ settings, running in $\mathcal{O}(Np \min(n, p))$ time where N is the number of samples drawn.

Note that our **BayesianLasso** package implements the modified Hans sampler via the function `Modified_Hans_Gibbs()`. In this function, $\mathbf{X}$ and $\mathbf{y}$ represent the covariate matrix and the response vector, respectively. The arguments `a1`, `b1`, `u1`, and `v1` correspond to the hyperparameters of the priors for σ^2 and λ^2 . The total number of MCMC iterations is specified by `nsamples`, and the initial values for $\boldsymbol{\beta}$, σ^2 and λ^2 are set via `beta_init`, `sigma2_init` and `lambda_init`, respectively. The argument `verbose` controls whether the sampling progress is printed during execution. The argument `thin` controls the thinning interval of the MCMC chain. Only every thin-th draw is stored. The default value is 1, corresponding to no thinning.

5.2 PC Gibbs sampler

Our modified PC Gibbs sampler is a modification of the Gibbs sampler developed in Park and Casella (2008) and is presented in Algorithm 2 and consists of a four-block Gibbs sampling scheme for estimating the model parameters, auxiliary variables (using the representation (2)), and the tuning

parameter λ^2 . Specifically, it involves sampling from the full conditional distributions: $[\beta|\mathcal{D}, \sigma^2, \lambda^2, \eta]$, $[\sigma^2|\mathcal{D}, \beta, \lambda^2, \eta]$, $[\lambda^2|\mathcal{D}, \beta, \sigma^2, \eta]$, and $[\eta|\mathcal{D}, \beta, \sigma^2, \lambda^2]$. The time complexity of Algorithm 2 consists of a one-time precomputation cost and a per-iteration cost. Precomputing quantities such as $\mathbf{X}^\top \mathbf{X}$, $\mathbf{X}^\top \mathbf{y}$, and $\|\mathbf{y}\|_2^2$ requires $\mathcal{O}(np^2)$ operations. Each MCMC iteration then involves sampling $\beta^{(i)}$ from a multivariate normal distribution, which requires computing a matrix square root—typically via Cholesky factorization or eigen-decomposition—and therefore incurs $\mathcal{O}(p^3)$ operations per iteration (Golub and van Loan, 2013). The total complexity is thus $\mathcal{O}(np^2 + Np^3)$, where N is the number of MCMC samples. A key advantage of this algorithm is that the per-iteration cost is independent of n , making it particularly attractive when n is large; however, the cubic scaling in p becomes dominant as the number of predictors increases.

In contrast, the modified Hans sampler updates β coordinate-wise, with each coordinate update requiring $\mathcal{O}(\min(n, p))$ operations. Updating all p coefficients therefore costs $\mathcal{O}(p \min(n, p))$ per iteration, leading to total complexity $\mathcal{O}(Np \min(n, p))$. Comparing per-iteration costs, the coordinate-wise updates scale as $\mathcal{O}(p \min(n, p))$, while the multivariate update in the PC sampler scales as $\mathcal{O}(p^3)$. As p increases, the cubic dependence in the multivariate approach becomes computationally expensive, whereas the coordinate-wise updates scale more favorably, particularly in high-dimensional settings. This complexity suggests that Algorithm 1 may be computationally more efficient than Algorithm 2 in high-dimensional settings, particularly when p is large relative to n . This theoretical scaling is consistent with the empirical runtimes reported in Table 5, where the Hans sampler exhibits clear computational advantages in higher-dimensional settings. In addition, for very large p , storing and factorizing the $p \times p$ Gram matrix may become the dominant memory and computational bottleneck, further favoring coordinate-wise approaches in ultra-high-dimensional problems.

Algorithm 2 Modified PC Gibbs sampler

```

1: Inputs:  $\mathbf{y}, \mathbf{X}, \tilde{a} > 0, \tilde{b} > 0, \tilde{u} > 0, \tilde{v} > 0, \lambda^2 > 0$ , and  $\sigma^{2(0)} > 0$ .
2:  $\eta^{(0)} = \mathbf{1}_n$ ;
3: for  $i = 1, \dots, N$  do
4:    $\mathbf{A} \leftarrow \text{diag}(\eta^{(i-1)})$ ;
5:    $\beta^{(i)} \sim N_p \left[ (\mathbf{X}^\top \mathbf{X} + \lambda^{2(i-1)} \mathbf{A})^{-1} \mathbf{X}^\top \mathbf{y}, \sigma^{2(i-1)} (\mathbf{X}^\top \mathbf{X} + \lambda^{2(i-1)} \mathbf{A})^{-1} \right]$ ;
6:    $\sigma^{2(i)} \sim \text{IG} \left( \tilde{a} + \frac{n+p}{2}, \tilde{b} + \frac{1}{2} \|\mathbf{y}\|_2^2 - \mathbf{y}^\top \mathbf{X} \beta^{(i)} + \frac{1}{2} (\beta^{(i)})^\top (\mathbf{X}^\top \mathbf{X} + \lambda^{2(i-1)} \mathbf{A}) \beta^{(i)} \right)$ ;
7:    $\lambda^{2(i)} \sim \text{Gamma} \left( \tilde{u} + \frac{p}{2}, \tilde{v} + \frac{(\beta^{(i)})^\top \mathbf{A} \beta^{(i)}}{2\sigma^{2(i)}} \right)$ ;
8:   for  $j = 1, \dots, p$  do
9:      $\eta_j^{(i)} \sim \text{Inverse-Gaussian} \left( \frac{\sigma^{(i)}}{\lambda |\beta_j^{(i-1)}|}, 1 \right)$ .
10:  end for
11: end for

```

Note that our **BayesianLasso** package implements the modified PC sampler via the function `Modified_PC_Gibbs()`. In this function, $\mathbf{X}$ and $\mathbf{y}$ represent the covariate matrix and the response vector, respectively. The arguments `a1`, `b1`, `u1`, and `v1` correspond to the hyperparameters of the priors for σ^2 and λ^2 . The total number of MCMC iterations is specified by `nsamples`, and the initial values for σ^2 and λ^2 are set via `sigma2_init` and `lambda_init`. The argument `verbose` controls whether the sampling progress is printed during execution. The argument `thin` controls the thinning interval of the MCMC chain. Only every `thin`-th draw is stored. The default value is 1, corresponding to no thinning. The MCMC outputs returned by `Modified_Hans_Gibbs()` and `Modified_PC_Gibbs()` are standard R matrices and vectors, which can be easily converted to objects used by common MCMC diagnostic packages such as `posterior` and `coda`. For example:

```
library(BayesianLasso)
fit <- Modified_Hans_Gibbs(X, y)
```

```
library(posterior)
draws <- as_draws_matrix(fit$mBeta)
```

Alternatively, the samples can be converted to objects used by the `coda` package:

```
library(coda)
```

```
mcmc_beta <- mcmc(fit$mBeta)
```

6 Performance comparison on simulated and benchmark datasets

Performance metrics. The impact of autocorrelation within the chains on estimation uncertainty can be quantified using the effective sample size (ESS). We compute ESS using the `ess_bulk()` function from the `posterior` package in R (Bürkner et al., 2023; Vehtari et al., 2021). To evaluate the efficiency of our proposed MCMC approach relative to the modified PC Gibbs sampler and the aforementioned R packages, we use the following metric

$$\text{Efficiency} = \frac{\text{ESS}}{\text{time}};$$

where `time` represents execution time in seconds. All runtimes are measured using `system.time()` to ensure consistent timing across samplers. Additionally, we assess whether the MCMC samples have reached a stationary distribution and exhibit adequate mixing using Gelman and Rubin's convergence diagnostic, $\hat{R}$ (Gelman and Rubin, 1992). We assess whether the outputs from each chain are indistinguishable by examining the scale reduction factor, considering values below 1.1 as an indication of convergence. We also evaluate the quality of mixing using the ratio

$$\text{Mix \%} = 100 \times \frac{\text{ESS}}{N}$$

where N represents the total number of samples.

We use weakly informative priors for the variance and shrinkage parameters by setting $\tilde{a} = \tilde{b} = \tilde{u} = \tilde{v} = 0.01$. These values correspond to diffuse Gamma priors on σ^2 and λ^2 , allowing the posterior inference to be driven primarily by the data. In practice, the algorithm is not highly sensitive to small changes in these hyperparameters, although extremely informative priors may influence both shrinkage strength and MCMC mixing. As a general guideline, small values (e.g., 0.01) provide weak regularization while maintaining stable posterior computation.

6.1 Simulation study

We conduct a controlled simulation study to compare the computational efficiency of the modified Hans sampler and the modified PC Gibbs sampler under a sparse linear regression setting.

Data-generating mechanism. We generate synthetic data from the linear model

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon},$$

where $\mathbf{X} \in \mathbb{R}^{n \times p}$ has independent standard normal entries, $\boldsymbol{\beta} = (2, 2, 2, 0, \dots, 0)^\top$ contains three non-zero coefficients and $p - 3$ zeros, and $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, \mathbf{I}_n)$. In our experiments we set $n = 100$ and $p = 10$. This configuration yields a moderately sparse setting with non-trivial shrinkage behaviour.

MCMC configuration. For both samplers we generate $N = 2000$ posterior samples with hyperparameters $(a_1, b_1, u_1, v_1) = (2, 1, 2, 1)$. The first 200 iterations are discarded as burn-in. The modified Hans sampler is initialized with $\boldsymbol{\beta}^{(0)} = \mathbf{1}$, $\lambda^{2(0)} = 1$, and $\sigma^{2(0)} = 1$, while the modified PC sampler uses identical hyperparameter settings and initial variance parameters for comparability.

Table 2 summarizes the simulation results for the modified Hans and modified PC Gibbs samplers. While both samplers achieve comparable mixing percentages across all parameters, the modified Hans sampler yields substantially larger effective sample sizes for $\boldsymbol{\beta}$, σ^2 , and λ^2 , as well as a shorter computation time. In particular, the efficiency of the Hans sampler exceeds that of the PC sampler by approximately 44% for $\boldsymbol{\beta}$ (256.73 versus 178.45), 81% for σ^2 (271.73 versus 149.97), and 151% for λ^2 (223.75 versus 89.03), while also requiring less computation time (0.05 seconds versus 0.09 seconds). Overall, these results demonstrate that the modified Hans sampler provides markedly improved sampling efficiency at a lower computational cost in the simulated setting.

Table 3 reports 95% credible intervals for the regression coefficients and hyperparameters in the simulated dataset with three nonzero coefficients ($\beta_1 = \beta_2 = \beta_3 = 2$) and the remaining coefficients equal to zero. For both the modified Hans and modified PC samplers, the 95% credible intervals for β_1, β_2 , and β_3 contain the true value 2, while the credible intervals for $\beta_4, \dots, \beta_{10}$ all contain 0. Consequently, the empirical coverage for $\boldsymbol{\beta}$ in this replicate is 1.00 overall, and also 1.00 when reported separately for nonzero and zero coefficients. The credible intervals for σ^2 and λ^2 are similar across

Dataset	Method	β	Eff	σ^2	Eff	λ^2	Eff	Time (s)
		Mix %	($\times 10^2$)	Mix %	($\times 10^2$)	Mix %	($\times 10^2$)	
Simulated	Hans	74.45	256.73	78.80	271.73	64.88	223.75	0.05
	PC	85.65	178.45	71.98	149.97	42.73	89.03	0.09

Table 2: Mixing percentages, effective sample sizes, and computation times (in seconds) for the simulated dataset using the Hans and PC Gibbs samplers from the BayesianLasso package. Efficiencies are reported in units of $\times 10^2$. Higher mixing percentages and efficiencies indicate improved sampling performance.

the two samplers, indicating comparable posterior uncertainty for these hyperparameters in this simulated setting. Since λ^2 is a prior hyperparameter rather than a parameter in the data-generating mechanism, no ground-truth value is available for coverage assessment in the simulation study.

Parameter	Hans		PC	
	95% CI (lo, hi)	Covered?	95% CI (lo, hi)	Covered?
β_1 (true = 2)	(1.72, 2.12)	Yes	(1.69, 2.12)	Yes
β_2 (true = 2)	(1.85, 2.17)	Yes	(1.86, 2.19)	Yes
β_3 (true = 2)	(1.81, 2.24)	Yes	(1.82, 2.24)	Yes
β_4 (true = 0)	(-0.16, 0.22)	Yes	(-0.16, 0.22)	Yes
β_5 (true = 0)	(-0.15, 0.24)	Yes	(-0.15, 0.22)	Yes
β_6 (true = 0)	(-0.07, 0.30)	Yes	(-0.06, 0.29)	Yes
β_7 (true = 0)	(-0.12, 0.27)	Yes	(-0.13, 0.27)	Yes
β_8 (true = 0)	(-0.11, 0.26)	Yes	(-0.11, 0.27)	Yes
β_9 (true = 0)	(-0.37, 0.01)	Yes	(-0.35, 0.02)	Yes
β_{10} (true = 0)	(-0.16, 0.19)	Yes	(-0.16, 0.18)	Yes
σ^2 (true = 1)	(0.81, 1.40)	Yes	(0.81, 1.42)	Yes
λ^2	(0.75, 4.72)	–	(0.75, 4.72)	–
Coverage (nonzero β)	1.00		1.00	
Coverage (zero β)	1.00		1.00	
Coverage (overall β)	1.00		1.00	

Table 3: Simulation study: 95% equal-tailed credible intervals (CIs) for regression coefficients and hyperparameters based on post-burn-in MCMC samples (iterations 201–2000). Coverage refers to whether the true parameter value lies within the reported 95% CI (reported as proportions for nonzero and zero coefficients).

Table 4 summarizes variable selection performance over 100 simulated datasets with three nonzero coefficients and seven zeros. Both samplers achieved perfect sensitivity (1.00) and zero false negative rate, indicating that all true signals were consistently selected across replicates. The overall selection accuracy was high for both methods (approximately 0.97), with specificity around 0.95, implying that occasional false positives occurred among truly zero coefficients. Consistent with this, the false discovery rate was low (about 0.07 on average) for both samplers.

Method	Accuracy	Sensitivity	Specificity	FDR	FNR
Hans	0.97 ± 0.06	1.00 ± 0.00	0.96 ± 0.08	0.07 ± 0.13	0.00 ± 0.00
PC	0.97 ± 0.06	1.00 ± 0.00	0.95 ± 0.09	0.07 ± 0.13	0.00 ± 0.00

Table 4: Simulation study (100 replicates): variable selection performance of the modified Hans and modified PC Gibbs samplers. A coefficient is declared *selected* if its 95% credible interval excludes zero. Reported values are mean $\pm$ standard deviation across replicates.

Out-of-sample predictive performance was evaluated using mean squared error (MSE) on an independently generated test dataset. The modified Hans and PC samplers achieved nearly identical predictive accuracy, with test MSE values of 1.077 and 1.075, respectively. Both values are close to the true noise variance ($\sigma^2 = 1$) used in the data-generating process, indicating that the fitted models provide well-calibrated predictions. The negligible difference between the two samplers suggests that, despite differences in sampling efficiency, their predictive performance in this simulated setting is essentially equivalent.

6.2 Benchmark experiments

In this section, we compare the performance of our modified Hans sampler with our modified PC Gibbs sampler, as well as the R packages `monomvn`, `bayeslm`, `rstan`, and `bayesreg` (Gramacy, 2024; He et al., 2022; Stan Development Team, 2020; Makalic and Schmidt, 2016). The comparison is based on several benchmark datasets that represent a diverse range of scenarios where $n > p$, $p > n$ and $p \gg n$. For $n > p$, we consider the Diabetes dataset with all pairwise interactions of the original variables (referred to as Diabetes²) from the `lars` package (Efron et al., 2004; Hastie and Efron, 2022), with $n = 442$ and $p = 55$; the Kakadu dataset with all pairwise interactions (Kakadu²) from the `Ecdat` package (Croissant and Graves, 2022), with $n = 1827$ and $p = 252$; and the Crime dataset from the UCI Machine Learning Repository with $n = 2215$ and $p = 98$ (Redmond, 2002).

We use 1,000 burn-in samples followed by 5,000 samples for inference. The computations were performed on an Apple M1 Pro with 12 cores and 16 GB of RAM. The Gibbs sampler methods were implemented in the R programming language (version 4.4.2), leveraging the computational efficiency of the `Rcpp` (version 1.0.13-1) (Eddelbuettel et al., 2024a; Eddelbuettel and François, 2011; Eddelbuettel, 2013; Eddelbuettel and Balamuta, 2018) and `RcppArmadillo` (version 14.2.2-1) (Eddelbuettel and Sanderson, 2014; Eddelbuettel et al., 2024b) packages. All Gibbs samplers developed here were run 5 times on all datasets and the results were averaged.

Table 5 presents the mixing percentages, sampling efficiencies, and elapsed times (in seconds) for the modified Hans and PC Gibbs samplers applied to the benchmark datasets Diabetes², Kakadu², and Crime. It is important to note that the Gelman–Rubin diagnostic $\hat{R}$ for each model parameter was below 1.01, indicating convergence, and the effective sample size (ESS) for β corresponds to the median of the ESS values for the p -dimensional vector β . The results indicate that the modified Hans sampler is the most efficient for sampling σ^2 and λ^2 in the Diabetes² and Kakadu² datasets, and also the most efficient for sampling β in both the Diabetes² and Kakadu² datasets. For the Crime dataset, the modified PC sampler achieves the highest efficiency for sampling β , while `bayeslm` achieves the highest efficiency for λ^2 , and the modified Hans sampler achieves the highest efficiency for σ^2 . Furthermore, the modified Hans sampler achieves the shortest computation time across all three datasets: Diabetes², Kakadu², and Crime. Note that `monomvn` was extremely slow on the Kakadu² dataset. Therefore, its output is reported as NA, as it was not within a comparable range of performance with the other samplers.

Dataset	Method	β		σ^2		λ^2		Time (s)
		Mix %	Eff ($\times 10^2$)	Mix %	Eff ($\times 10^2$)	Mix %	Eff ($\times 10^2$)	
Diabetes ²	Hans	26.91	50.77	75.62	142.68	27.13	51.20	0.21
	PC	78.21	38.34	74.80	36.67	15.73	7.71	0.81
	monomvn	98.14	0.72	97.08	0.71	96.11	0.71	59.21
	bayeslm	12.28	10.24	50.43	42.27	4.29	3.58	0.48
	rstan	98.50	0.57	94.55	0.55	99.21	0.58	68.51
	bayesreg	77.26	14.59	61.62	11.59	17.52	3.33	2.15
Kakadu ²	Hans	19.29	3.22	78.85	13.16	5.51	0.92	2.39
	PC	85.64	0.76	72.99	0.65	7.88	0.07	44.70
	monomvn	NA	NA	NA	NA	NA	NA	NA
	bayeslm	15.13	1.80	52.54	3.72	2.46	0.17	6.16
	rstan	98.45	0.12	98.43	0.12	96.49	0.12	347.10
	bayesreg	84.65	1.98	69.65	1.63	4.34	0.10	17.08
Crime	Hans	6.02	5.93	9.42	9.28	0.24	0.24	0.40
	PC	83.65	9.89	24.22	2.86	12.11	1.43	3.38
	monomvn	97.76	0.04	97.38	0.04	98.76	0.04	862.59
	bayeslm	5.37	2.19	13.94	5.65	3.79	1.53	0.98
	rstan	98.07	0.55	92.44	0.52	92.39	0.52	70.81
	bayesreg	81.84	9.09	29.38	3.27	13.62	1.51	3.62

Table 5: Mixing percentages, efficiencies, and computation times (in seconds) for each dataset using the Hans and PC Gibbs samplers from the `BayesianLasso` package, as well as the R packages `monomvn`, `bayeslm`, `rstan`, and `bayesreg`.

For these datasets, the modified Hans sampler exhibits lower mixing percentages for β relative to σ^2 . This is consistent with the coordinate-wise structure of the algorithm, where regression coefficients are updated sequentially and mixing can be slower in the presence of strong posterior dependence

among predictors. In contrast, σ^2 is a scalar global parameter whose full conditional depends on aggregate quantities — namely, the residual sum of squares and $\|\beta\|_1$ — resulting in substantially faster mixing.

Table 6 presents results for two high-dimensional settings in which the number of predictors exceeds the number of observations ($p > n$). Specifically, the Eye dataset contains $n = 120$ observations and $p = 200$ predictors, while the Riboflavin dataset has $n = 71$ observations and $p = 4088$ predictors. These settings illustrate the performance of the samplers under challenging high-dimensional regimes, particularly when $p \gg n$. Here, we use 1,000 burn-in samples followed by 10,000 samples for inference. We report results for the Hans and PC Gibbs samplers implemented in the BayesianLasso package, as well as for the bayesreg package. Other samplers and R packages were not included due to their prohibitively high computational cost in these settings, resulting in runtimes that were not directly comparable. Notably, the PC sampler was extremely slow on the Riboflavin dataset; consequently, its results are reported as NA, as they were not obtained within a comparable computational budget. This behaviour is expected, as the computational cost of the PC sampler scales cubically with p , that is, $\mathcal{O}(np^2 + Np^3)$.

Dataset	Method	β		σ^2		λ^2		Time (s)
		Mix %	Eff ($\times 10^2$)	Mix %	Eff ($\times 10^2$)	Mix %	Eff ($\times 10^2$)	
Eye	Hans	25.14	19.07	7.44	5.64	2.96	2.25	1.32
	PC	78.28	2.89	5.97	0.21	3.71	0.13	27.44
	bayesreg	79.28	2.89	5.90	0.31	3.33	0.17	19.08
Riboflavin	Hans	9.67	0.70	0.13	0.01	0.05	0.00	13.95
	PC	NA	NA	NA	NA	NA	NA	NA
	bayesreg	96.33	0.41	0.08	0.00	0.07	0.00	234.08

Table 6: Mixing percentages, efficiencies, and computation times (in seconds) for the Eye and Riboflavin datasets using the Hans and PC Gibbs samplers from the BayesianLasso package, and the bayesreg package.

Based on the results shown in Table 6, the modified Hans sampler generally exhibits substantially higher efficiency and markedly shorter computation times compared with the competing methods. For the Eye dataset ($n = 120$, $p = 200$), the Hans sampler attains the largest efficiency across all parameters while maintaining the shortest runtime. In contrast, the PC sampler and the bayesreg implementation yield considerably lower efficiencies and require longer computation times. These differences become more pronounced for the Riboflavin dataset ($n = 71$, $p = 4088$). As the dimensionality increases, the PC sampler becomes computationally infeasible, while bayesreg incurs a substantial computational cost of 234.08 seconds. In comparison, the modified Hans sampler maintains higher efficiency for β and σ^2 , together with a substantially lower runtime of 13.95 seconds. Overall, these results indicate that the Hans sampler is more computationally efficient and scalable in high-dimensional settings, making it well suited for Bayesian Lasso regression when $p \gg n$.

7 Discussion

This paper introduces the **BayesianLasso** R package, which provides a comprehensive and computationally efficient implementation of Bayesian Lasso regression based on a newly defined Lasso distribution. The package encapsulates recent methodological advances by offering implementations of both the modified Hans and PC Gibbs samplers, tailored to exploit the distributional structure of the Lasso prior.

Central to the package is the formal development of the Lasso distribution, which we establish as a member of the exponential family. We derive key theoretical properties—including the probability density function, cumulative distribution function, moments, and a numerically stable inverse-CDF sampling method—which underpin the samplers implemented in the package. In particular, the Lasso distribution serves as the full conditional distribution for regression coefficients in the proposed Gibbs sampling framework.

The **BayesianLasso** package is designed with usability and extensibility in mind, offering an accessible interface for researchers and practitioners to fit Bayesian Lasso models in regression settings. The package also provides utility functions for working directly with the Lasso distribution, including density evaluation, random sampling, and cumulative probability calculations.

Our empirical evaluations demonstrate that the modified Hans sampler achieves strong computational efficiency and favorable mixing behavior across both low-dimensional and high-dimensional

settings. In the benchmark datasets where $n > p$ (Diabetes², Kakadu², and Crime), the results show that the modified Hans sampler attains substantially higher sampling efficiency for all parameters in the Diabetes² and Kakadu² datasets. In the Crime dataset, it achieves the highest efficiency for σ^2 and the shortest computation time, though the PC sampler and `bayeslm` attain higher efficiency for β and λ^2 , respectively. Overall, the modified Hans sampler consistently achieves the shortest computation times across all three datasets, indicating strong computational scalability relative to existing implementations.

Similar patterns are observed in high-dimensional settings ($p > n$), represented by the Eye and Riboflavin datasets. In these scenarios, the modified Hans sampler demonstrates substantially higher sampling efficiency across all parameters in the Eye dataset, while also achieving markedly lower computation times compared with alternative implementations. For the Riboflavin dataset, the PC sampler becomes computationally infeasible, and while `bayesreg` remains applicable, it incurs a substantially higher computational cost. The modified Hans sampler maintains higher efficiency for β and σ^2 alongside a markedly shorter runtime, highlighting its robustness and scalability in challenging high-dimensional regimes where efficient posterior exploration is critical.

Overall, the empirical results suggest that the modified Hans sampler provides an effective and computationally scalable approach for posterior inference in Bayesian Lasso models. By integrating the method into an easy-to-use R package, we aim to facilitate its adoption and support broader use of Bayesian Lasso methodology in applied statistical analysis.

The Gibbs-based approach developed in this paper is particularly attractive when the model admits structured conditional distributions that can be exploited for efficient updates, as in the Gaussian Bayesian Lasso considered here. Compared to Hamiltonian Monte Carlo (HMC) as implemented in Stan, the proposed sampler avoids gradient evaluations and matrix factorizations at each iteration, leading to substantially lower per-iteration computational cost in high-dimensional settings. While HMC can exhibit superior mixing for strongly correlated parameters due to joint updates, its computational burden scales cubically in the number of predictors for dense regression models.

Relative to variational inference, the Gibbs sampler provides asymptotically exact posterior samples rather than approximate solutions. Although variational methods are typically faster, they may underestimate posterior uncertainty, particularly in hierarchical shrinkage models where strong posterior dependence is present. Therefore, the proposed Gibbs sampler is especially preferable in moderate- to high-dimensional Gaussian regression problems where exact posterior inference is desired and structured conditional updates can be exploited.

The current implementation focuses on Gaussian linear regression, where the conditional distribution of each regression coefficient can be expressed in terms of the Lasso distribution introduced in this paper, enabling efficient Gibbs updates. Extensions to generalized linear models (GLMs) are conceptually possible by combining the proposed Lasso distribution with suitable data augmentation schemes, so that conditional updates remain tractable within a Gibbs framework.

Extensions to related penalization schemes such as the elastic net or group Lasso are also feasible through modifications of the prior structure. The elastic net can be obtained by combining ℓ_1 and ℓ_2 penalties via an additional Gaussian shrinkage component, while the group Lasso requires block-wise shrinkage priors defined on groups of coefficients. In these cases, the general Gibbs sampling framework remains applicable, although the form of the conditional distributions and the associated computational complexity would differ. These extensions represent promising directions for future development of the `BayesianLasso` package.

8 Code availability

The R package `BayesianLasso` implementing the methods described in this paper is available on CRAN at <https://CRAN.R-project.org/package=BayesianLasso> and on GitHub at <https://github.com/garthtarr/BayesianLasso>.

9 Competing interests

The authors declare that they have no conflict of interest.

Acknowledgments

The following source of funding is gratefully acknowledged: Australian Research Council Discovery Project grant (DP210100521).

References

- P.-C. Bürkner, J. Gabry, M. Kay, and A. Vehtari. posterior: Tools for working with posterior distributions, 2023. URL <https://mc-stan.org/posterior/>. R package version 1.4.1. [p11]
- Y. Croissant and S. Graves. *Ecdat: Data Sets for Econometrics*, 2022. URL <https://CRAN.R-project.org/package=Ecdat>. R package version 0.4-2. [p13]
- D. Eddelbuettel. *Seamless R and C++ Integration with Rcpp*. Springer, New York, 2013. doi: 10.1007/978-1-4614-6868-4. ISBN 978-1-4614-6867-7. [p13]
- D. Eddelbuettel and J. J. Balamuta. Extending R with C++: A Brief Introduction to Rcpp. *The American Statistician*, 72(1):28–36, 2018. doi: 10.1080/00031305.2017.1375990. [p13]
- D. Eddelbuettel and R. François. Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, 40(8):1–18, 2011. doi: 10.18637/jss.v040.i08. [p13]
- D. Eddelbuettel and C. Sanderson. Rcpparmadillo: Accelerating R with high-performance C++ linear algebra. *Computational Statistics and Data Analysis*, 71:1054–1063, March 2014. doi: 10.1016/j.csda.2013.02.005. [p13]
- D. Eddelbuettel, R. Francois, J. Allaire, K. Ushey, Q. Kou, N. Russell, I. Ucar, D. Bates, and J. Chambers. *Rcpp: Seamless R and C++ Integration*, 2024a. URL <https://CRAN.R-project.org/package=Rcpp>. R package version 1.0.13-1. [p13]
- D. Eddelbuettel, R. Francois, D. Bates, B. Ni, and C. Sanderson. *RcppArmadillo: 'Rcpp' Integration for the 'Armadillo' Templated Linear Algebra Library*, 2024b. URL <https://CRAN.R-project.org/package=RcppArmadillo>. R package version 14.2.2-1. [p13]
- B. Efron, T. Hastie, I. Johnstone, and R. Tibshirani. Least angle regression. *The Annals of Statistics*, 32(2):407 – 499, 2004. doi: 10.1214/009053604000000067. URL <https://doi.org/10.1214/009053604000000067>. [p13]
- A. Gelman and D. B. Rubin. Inference from iterative simulation using multiple sequences. *Statistical science*, 7(4):457–472, 1992. URL <https://doi.org/10.1214/ss/1177011136>. [p11]
- G. H. Golub and C. F. van Loan. *Matrix Computations*. The Johns Hopkins University Press, fourth edition, 2013. ISBN 9781421407944. URL <https://doi.org/10.56021/9781421407944>. [p10]
- R. B. Gramacy. *monomvn: Estimation for MVN and Student-t Data with Monotone Missingness*, 2024. URL <https://CRAN.R-project.org/package=monomvn>. R package version 1.9-21. [p13]
- C. Hans. Bayesian lasso regression. *Biometrika*, 96(4):835–845, 2009. URL <https://doi.org/10.1093/biomet/asp047>. [p1, 2, 8, 9]
- T. Hastie and B. Efron. *lars: Least Angle Regression, Lasso and Forward Stagewise*, 2022. URL <https://CRAN.R-project.org/package=lars>. R package version 1.3. [p13]
- T. Hastie, R. Tibshirani, and M. Wainwright. *Statistical Learning with Sparsity: The Lasso and Generalizations*. Chapman & Hall/CRC, 2015. ISBN 1498712169. URL <https://hastie.su.domains/StatLearnSparsity/>. [p8]
- J. He, P. R. Hahn, H. Lopes, and A. Herren. *bayeslm: Efficient Sampling for Gaussian Linear Regression with Arbitrary Priors*, 2022. URL <https://CRAN.R-project.org/package=bayeslm>. R package version 1.0.1. [p13]
- E. Makalic and D. F. Schmidt. High-dimensional Bayesian regularised regression with the bayesreg package. *arXiv preprint arXiv:1611.06649*, 2016. URL <https://arxiv.org/abs/1611.06649>. [p13]
- G. Marsaglia. Evaluating the normal distribution. *Journal of Statistical Software*, 11:1–11, 2004. URL <https://doi.org/10.18637/jss.v011.i04>. [p4]
- J. P. Mills. Table of the ratio: Area to bounding ordinate, for any portion of normal curve. *Biometrika*, 18(3-4):395–400, 1926. URL <https://doi.org/10.1093/biomet/18.3-4.395>. [p1]
- M. Mächler. Asymptotic Tail Formulas For Gaussian Quantiles, 2022. URL <https://cran.r-project.org/package=DPQ/vignettes/qnorm-asymp.pdf>. [p5]
- R. M. Neal. Slice sampling. *The Annals of Statistics*, 31(3):705–767, 2003. URL <https://doi.org/10.1214/aos/1056562461>. [p8]

- J. Ormerod, M. J. Davoudabadi, G. Tarr, and S. Mueller. BayesianLasso: Bayesian Lasso Regression and Tools for the Lasso Distribution, 2025. URL <https://github.com/garhtarr/BayesianLasso>. R package on GitHub. [p3]
- T. Park and G. Casella. The Bayesian Lasso. *Journal of the American Statistical Association*, 103(482): 681–686, 2008. URL <https://doi.org/10.1198/016214508000000337>. [p1, 2, 8, 9]
- M. Redmond. Communities and Crime. UCI Machine Learning Repository, 2002. URL <https://doi.org/10.24432/C53W3X>. [p13]
- R. Reemtsen. Modifications of the first Remez algorithm. *SIAM Journal on Numerical Analysis*, 27(2): 507–518, 1990. URL <https://doi.org/10.1137/0727031>. [p5]
- Stan Development Team. RStan: the R interface to Stan, 2020. URL <http://mc-stan.org/>. [p13]
- Statisticat and LLC. *LaplaceDemon: Complete Environment for Bayesian Inference*, 2021. URL <https://web.archive.org/web/20150206004624/http://www.bayesian-inference.com/software>. R package version 16.1.6. [p1]
- R. Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 58(1):267–288, 1996. URL <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>. [p1, 2]
- A. Vehtari, A. Gelman, D. Simpson, B. Carpenter, and P.-C. Bürkner. Rank-normalization, folding, and localization: An improved rhat for assessing convergence of MCMC (with discussion). *Bayesian Analysis*, 2021. URL <https://doi.org/10.1214/20-BA1221>. [p11]

John T. Ormerod
School of Mathematics and Statistics, University of Sydney
Sydney
Australia
john.ormerod@sydney.edu.au

Mohammad Javad Davoudabadi
School of Mathematics and Statistics, University of Sydney
Sydney
Australia
mohammad.davoudabadi@sydney.edu.au

Garth Tarr
School of Mathematics and Statistics, University of Sydney
Sydney
Australia
garth.tarr@sydney.edu.au

Samuel Muller
Faculty of Science and Engineering, Macquarie University
Sydney
Australia
samuel.muller@mq.edu.au

Jonathon Tidswell
School of Mathematics and Statistics, University of Sydney
Sydney
Australia
jonathon.tidswell@sydney.edu.au